

Phase 5: Final Comparison

Overview

This phase implements XGBoost — the candidate selected in Phase 4 — and compares it head-to-head against our best Phase 3 model on the same 5-class predictive maintenance problem. Both models are evaluated on an identical 80/20 stratified test split with the same SMOTE-balanced training data, ensuring a fair comparison. The report covers both quantitative results and a qualitative discussion of why each model performed the way it did.

Experimental Setup

Both models were trained and evaluated under identical conditions:

- **Dataset:** AI4I 2020, 5-class formulation (No Failure, TWF, HDF, PWF, OSF — RNF excluded)
- **Split:** 80/20 stratified, random seed 42
- **Preprocessing:** StandardScaler fitted on training data only
- **Imbalance handling:** SMOTE applied to training set, producing 38,605 balanced samples (7,721 per class)
- **Test set:** 1,997 samples, untouched throughout training

Model A — Random Forest (our Phase 3 best model): 300 trees, max depth 15, class_weight='balanced', trained on SMOTE-resampled data.

Model B — XGBoost (Phase 4 literature candidate): 300 estimators, max depth 6, learning rate 0.1, multi:softmax objective for 5-class classification, trained on the same SMOTE-resampled data with balanced sample weights.

Quantitative Results

Overall Metrics

Metric	Random Forest	XGBoost	Winner
Accuracy	0.9664	0.9750	XGBoost
Binary Recall	0.8333	0.8485	XGBoost
Binary Precision	0.5046	0.6087	XGBoost
Binary F1	0.6286	0.7089	XGBoost
Macro F1	0.7035	0.7223	XGBoost

XGBoost outperforms Random Forest on every aggregate metric. The most meaningful differences are in binary F1 (+0.080) and binary precision (+0.104) — XGBoost raises fewer false alarms while catching a slightly higher fraction of real failures.

Per-Class F1 Scores

Class	RF F1	XGB F1	Winner
No Failure	0.9830	0.9880	XGBoost
TWF	0.1200	0.0500	RF
HDF	0.7755	0.9565	XGBoost
PWF	0.8889	0.9143	XGBoost
OSF	0.7500	0.7027	RF

XGBoost wins on three of five classes, and its advantage on HDF is dramatic — 0.9565 vs 0.7755. The two classes where RF wins are the smallest ones: TWF (only 9 test samples) and OSF (16 test samples). Random Forest's SMOTE-driven aggressive oversampling helps it squeeze more recall out of the rarest classes, while XGBoost's sequential correction mechanism is more effective at cleanly separating the physically well-defined failure types like HDF and PWF.

Comparison to Literature

Safi et al. (2025) reported XGBoost achieving macro-F1 of 0.842 on a 7-class formulation without SMOTE. Our XGBoost with SMOTE achieves 0.7223. This is consistent with Safi et al.'s finding that adding oversampling to XGBoost tends to reduce its macro-F1 from 0.842 to around 0.762, because SMOTE artificially inflates minority class representation in ways that conflict with XGBoost's sequential error correction. Our number (0.7223) falls in that same range, confirming our implementation is behaving as expected.

Qualitative Comparison

Random Forest and XGBoost are both tree-based ensembles, but they work very differently. Random Forest builds all 300 trees in parallel, each on a random subset of data, and combines predictions by majority vote. XGBoost builds trees sequentially — each new tree focuses on the samples that previous trees got wrong, progressively correcting residual errors.

This difference explains most of the results. For HDF and PWF, which have sharp and consistent decision boundaries (one is defined by temperature differential at high speed, the other by torque × speed leaving a safe range), XGBoost's sequential refinement finds those boundaries far more precisely than independent majority voting. That's why it dominates these two classes.

For TWF and OSF — the rarest classes with the fewest samples — Random Forest benefits more from SMOTE because its voting mechanism directly counts synthetic minority examples on equal footing with real ones. XGBoost's gradient signal for these tiny classes is diluted by the dominant No Failure class even after oversampling, so its sequential correction keeps drifting toward the larger classes.

XGBoost's binary precision of 0.6087 versus RF's 0.5046 is practically important. It means that for every 10 failure alerts XGBoost raises, about 6 are genuine — compared to only 5 out of 10 for RF. In a real maintenance system, every false alarm means production engineers investigate a machine that is actually fine, which costs time and money. XGBoost's higher precision would result in fewer wasted investigations. Binary recall is nearly identical (0.8485 vs 0.8333), so XGBoost is catching roughly the same number of real failures while raising fewer false ones — a strictly better operating point for most industrial deployments.

Despite losing on aggregate metrics, Random Forest has two genuine advantages. First, it is more interpretable in the way it handles rare classes — the SMOTE + majority voting approach has a clear, intuitive story: generate more examples of rare failures and let each tree vote. Second, if the deployment context places extreme importance on never missing any TWF event (for example, if a tool wear failure causes catastrophic equipment damage), RF's higher TWF recall of 0.33 versus XGBoost's 0.11 may be decisive. The right choice ultimately depends on the cost structure of the real maintenance environment.

Summary

XGBoost is the stronger model for this problem by most meaningful measures. It achieves higher macro F1 (0.7223 vs 0.7035), higher binary F1 (0.7089 vs 0.6286), and meaningfully better precision on failure detection (0.6087 vs 0.5046). Its advantage is clearest on HDF and PWF — the two failure types with sharp, physically well-defined boundaries that XGBoost's sequential boosting exploits effectively. Random Forest retains an edge on TWF and OSF, where SMOTE's synthetic oversampling gives the majority-vote ensemble a slight benefit for the rarest classes.

The results are consistent with the literature: Safi et al. (2025) found XGBoost to be the top-performing model on AI4I 2020, and our comparison reproduces that finding under a controlled experimental setup. Both models fall well short of the inflated binary-detection scores (>0.995) reported by Waghulde et al. (2025) — but as discussed in Phase 4, those numbers come from a simpler binary task on a synthetic dataset, and our 5-class formulation is a more realistic and challenging test of what a maintenance system actually needs to do.